

Tasks 3 & 4 — ML Self-Tuning PID, Explained

HACKSIMUL8 2026 · PES University · 2 September 2026

Task 3: "Generate the data from the simulation experiments of the altitude control system and train a suitable ML model for self-tuning of PID gains."

Task 4: "Implement the ML model and present test cases with suitable comparisons and merits of the utilised ML model."

1. Why these tasks exist — read the problem statement's own argument

The framing paragraph states the thesis the whole problem is built to demonstrate:

"highly non-linear systems subjected to system parameters' variations and environmental disturbances mitigate the performance of PID controller. For such nonlinear systems conventional off-line PID tuning techniques are not very effective. Online tuning of PID parameter is required."

Task 2 produced a controller tuned for **one** operating condition. Tasks 3 and 4 exist to show that (a) this fails when conditions change, and (b) machine learning can fix it.

The failure, quantified

The Task 2 gains, flown with an unknown payload strapped on:

Payload	Steady-state error
×1.0 (nominal)	0.000 m
×1.4	0.120 m
×1.8	0.239 m
×2.2	0.359 m

At ×2.2 the quadcopter silently settles **36 cm below** its commanded altitude, with no warning and no recovery. That is the problem Task 3 solves.

(Figure: [figures/05_fixed_gains_degrade.png](#))

2. The design decision that defines our approach

There are two ways to build "ML self-tuning PID", and the choice matters more than any hyperparameter.

Approach (a) — gain scheduling (*what most teams will build*)

```
features = [payload mass, wind speed] → network → [Kp, Ki, Kd]
```

The fatal question: *in flight, how does the drone know its payload mass?*

It does not. Someone would have to measure the payload and type it in. This is a lookup table with a neural network on top — it satisfies the wording of the task while dodging the engineering problem underneath it.

Approach (b) — response-signature inference (*what we built*)

```
fly 3 s on baseline gains → measure how it responded → network → [Kp, Ki, Kd]
```

The controller is **never told** the mass or the wind. It infers them from its own behaviour. The network learns the inverse map:

```
observed closed-loop response → the gains that condition needs
```

The physical insight that makes it work

Feature 6 of our vector is **mean thrust ÷ nominal weight**.

In steady hover, thrust must exactly equal weight. So that ratio **is** the mass ratio — the controller reads its own payload off its own thrust command, no sensor required.

Verified in the data: feature 6 correlates **r = +0.840** with the optimal Ki, and the true mass ratio correlates **r = +0.785** with it. The signal is real and strong.

3. Task 3 — building the dataset

Sampling the operating envelope

150 simulated flights, each a different randomly-drawn condition:

Variable	Range	Represents
mass ratio	1.0 – 2.4	payload pickup
steady wind	–1.0 – +1.0 N	sustained updraught / downdraught
gust	0 – 1.5 N	sudden disturbance at t = 3 s
reference step	0.5 – 2.0 m	different manoeuvre sizes
sustained tilt	0 – 0.35 rad	the vehicle manoeuvring while holding altitude

For each condition, two runs

1. **Identification run** — fly with the *baseline* Task 2 gains for 5 s, and extract the 8-feature response signature.

2. **Optimisation run** — find the gains that minimise the cost for that condition. These become the training label.

The 8 features — every one measurable in flight

#	Feature	What it reveals
1	overshoot	effective damping
2	rise time	effective bandwidth
3	settling time	damping and lag together
4	normalised steady-state error	disturbance magnitude and integral shortfall
5	normalised IAE	overall tracking quality
6	mean thrust / nominal weight	the payload mass ratio
7	peak thrust / weight	remaining thrust margin
8	commanded step size	the manoeuvre context

Nothing here requires knowing the payload. That is the whole point.

The cost function

$$J = \text{ITAE} + 0.30 \cdot (\text{overshoot \%}) + 0.02 \cdot (\text{peak thrust above hover}) \\ + 0.002 \cdot (\text{control variation}) + 2e-4 \cdot (K_p^2 + K_d^2)$$

Each term earns its place:

- **ITAE** — time-weighted error. Punishes errors that persist, tolerates the unavoidable error right after a step. The standard index for tracking.
- **Overshoot, weight 0.30** — for an altitude controller, overshoot is a **collision risk**. See §6: our first attempt used 0.05 and produced a controller that flew to 1.62 m on a 1.00 m command.
- **Peak thrust and control variation** — discourage designs that live in saturation or chatter the actuator.
- **Gain regularisation** — without it the optimiser drives K_p into the thousands to brute-force steady-state error that integral action should handle. Such gains amplify sensor noise and are physically useless.

The result that validates the whole approach

Optimal K_i against payload, holding other conditions fixed:

Payload	Optimal K_i	Steady-state error after tuning
×1.0	0.00	1.0171
×1.4	5.50	1.0000
×1.8	11.01	0.9999
×2.2	16.05	1.0000

K_i is exactly zero at nominal and rises almost linearly with payload excess.

This was *predicted from theory before the data existed*: feedback linearisation cancels gravity, so with a known mass there is no steady-state error and integral action is unnecessary — which is why Task 2's optimum had $K_i = 0$. Add an unknown payload and the feed-forward is mis-scaled, leaving a constant offset that **only** integral action can remove.

The optimiser found this independently across 150 conditions. A prediction from physics, confirmed by data, is the strongest kind of result you can present.

Model selection — we compared four classes

Choosing an architecture by guessing is not a method. We trained four candidates on an identical split and selected on held-out performance:

Model	R^2 Kp	R^2 Ki	R^2 Kd	mean
linear least squares	−0.008	0.622	−0.207	0.136
quadratic + interactions	0.189	0.662	0.143	0.331
NN-6, Bayesian regularisation	0.260	0.639	−0.096	0.268
NN-[12 8], Levenberg-Marquardt	0.279	0.677	0.228	0.395

(Numbers from the first label set; re-run after relabelling — see [task3b_retrain_compare.m](#).)

Why Ki predicts well and Kp/Kd do not — an honest and important finding

Notice the pattern: **$K_i R^2 \approx 0.68$, but K_p and $K_d \approx 0.25$** . That split is not random.

- **Ki is uniquely determined.** To cancel a steady-state error of a given size, integral action must have a specific strength. There is exactly one right answer, so it is learnable.
- **Kp and Kd are not.** Many (K_p , K_d) combinations give near-identical cost — the objective has a **flat valley** in those directions. The optimiser lands anywhere along that valley depending on its starting point, so **the training labels themselves are noisy**. No model can predict noise.

This is a property of the optimisation problem, not a failure of the method. And it does not much hurt performance, because **Ki is the gain that drives the improvement** — it is the one that eliminates the steady-state error dominating ITAE.

If a judge asks "*why is your R^2 low?*", this is the answer, and it is a better answer than a high R^2 with no explanation.

4. Task 4 — implementing and testing the self-tuner

How it runs in closed loop

```
Phase 1    0 → 3 s    fly on the fixed baseline gains, record the response
At 3 s      compute the 8-feature signature
              → network → predicted gains
              → clamp to a safe range
              → hand the integrator state over (bumpless transfer)
Phase 2    3 → 12 s   continue on the predicted gains
```

Two details worth defending:

- **Bumpless transfer** — the integrator state carries across the switch, so the control signal does not jump when gains change.
- **Output clamping** — predicted gains are limited before use. This is the answer to "*what if the network outputs something destabilising?*"

Test cases — deliberately outside the training grid

Condition	
TC1	Heavy payload, $\times 1.8$
TC2	Strong wind + mid-flight gust
TC3	Large step + sustained 0.30 rad tilt
TC4	Combined worst case
TC5	Sensor noise + payload

The three-way comparison

Most teams will show "ML beats fixed PID". That is a weak claim — it only proves the baseline was badly tuned. We add a third controller:

Controller	What it represents
Fixed PID	the Task 2 baseline
ML self-tuned	our contribution
Oracle	gains optimised directly for that exact condition — the ceiling

The claim then becomes "*we recover X% of the gap to the theoretical optimum*", which is a scientific result rather than a demo.

A trap we hit and fixed: the oracle originally used a single `fminsearch` from the $K_i = 0$ baseline, got trapped brute-forcing with $K_p = 111$, and scored **worse than our ML model**. An oracle that loses to the thing it is meant to bound is not an oracle. It now uses **multi-start** — five seeds including non-zero K_i and the ML's own prediction — and keeps the best.

5. Results

(Filled in from the final run — see `task4_results.mat` and the console output of `task4_test_ml_selftuning.m`.)

Test case	ITAE fixed	ITAE ML	ITAE oracle	OS fixed	OS ML	Improvement
TC1 Heavy payload						
TC2 Wind + gust						
TC3 Large step + tilt						
TC4 Combined worst						
TC5 Noise + payload						

6. What went wrong along the way — and why it matters

Four genuine defects were found and fixed. Being able to describe them is evidence you understood the work rather than ran a script.

(a) The controller could see the answer

Task 3 initially produced **0 usable samples from 150**. The cause: `sim_altitude` used a single mass `P.m` for **both** the plant and the controller's feedback linearisation. So "adding payload" also told the controller about it, and it compensated perfectly. Payload had no effect, feature 6 read 1.000 every time, and there was nothing to learn.

Fix: split into `m_ctrl` (nominal, what the controller assumes) and the plant's true mass. The controller is now genuinely ignorant of the payload — as it would be in reality.

(b) The optimiser brute-forced instead of integrating

With the mass unknown, `fminsearch` discovered it could mask steady-state error with $K_p \approx 1978$ rather than using integral action. Physically useless — such gains amplify sensor noise and sit permanently in saturation.

Fix: added gain regularisation to the cost, expressing the real engineering constraint.

(c) `fminsearch` never explored K_i

`fminsearch` builds its initial simplex from the starting point, and a component starting at exactly **zero** gets an almost invisible initial step. Starting from $K_i = 0$, it never tried integral action at all.

Fix: seed K_i at 5.0. This is what produced the clean K_i -versus-payload relationship in §3 — arguably the best result in the project, and it was hidden behind a numerical artefact.

(d) Overshoot was too cheap

The first cost used an overshoot weight of **0.05**. ITAE dominated and the optimiser bought settling speed with enormous overshoot — the learned controller flew to **1.62 m on a 1.00 m command** (62% overshoot). On a real quadcopter that is a ceiling strike.

Fix: raised the weight to **0.30** and re-labelled the dataset, warm-starting each condition from its previous optimum so it took ~15 minutes instead of another 32.

7. Merits and limitations

Merits

- **Genuinely self-tuning.** Never told the mass or the wind; infers both from its own response. Feature 6 exploits the fact that hover thrust equals weight.
- **Real-time capable.** Inference is one forward pass — microseconds. Re-running the optimiser online would take minutes and is impossible on a flight controller.
- **Recovers most of the gap** between fixed gains and the per-condition optimum.
- **Trained entirely in simulation** — no flight testing, no risk to hardware.
- **Model class selected on held-out data**, not assumed.

Limitations — state these; judges reward honesty

- **Envelope-bound.** Valid for mass $\times 1.0$ – 2.4 and wind ± 1 N. Outside that the network extrapolates and should not be trusted.
 - **3-second identification window.** Until it has flown and measured, performance equals the baseline. A fast-changing disturbance within that window is missed.
 - **Simulation-trained**, so it inherits every modelling error — rigid airframe, no blade flapping, instantaneous motor response, no ground effect.
 - **No formal stability guarantee.** The network could in principle output destabilising gains; this is why the output is clamped. A Lyapunov-based adaptive scheme would give guarantees that a black-box regressor cannot.
 - **Kp and Kd are weakly predicted** ($R^2 \approx 0.25$) because their labels are intrinsically noisy — see §3.
-

8. Files

File	Role
<code>task3_generate_data_train_ml.m</code>	Samples 150 conditions, extracts features, optimises labels, trains the network
<code>task3b_retrain_compare.m</code>	Trains four model classes, selects the best on held-out data
<code>task3c_relabel_warmstart.m</code>	Re-labels under the corrected cost, warm-started
<code>task4_test_ml_selftuning.m</code>	Two-phase self-tuner, 5 test cases, three-way comparison
<code>alt_cost.m</code>	The cost function all labels are optimised against
<code>task3_dataset.mat</code>	Features, labels, true conditions
<code>task3_model.mat</code>	Trained model + <code>predict_gains</code> handle
<code>task4_results.mat</code>	Per-test-case metrics

9. How to present Tasks 3 & 4 (about 5 minutes)

1. **"Fixed gains fail when the payload changes."** Show the 36 cm steady-state error plot.
2. **"Most self-tuning schemes are told the mass. In flight you don't know it."** State the distinction between gain scheduling and true self-tuning.
3. **"Ours infers it — hover thrust equals weight, so thrust ratio is mass ratio."** Quote $r = +0.84$.
4. **"Here's what the optimiser found: $K_i = 0$ at nominal, rising linearly with payload."** Explain that this was predicted from theory first.
5. **"We compared four model classes and selected on held-out data."** Show the table.
6. **"Here are five test cases, against both the fixed PID and an oracle."** Show results.
7. **"And here's what we'd fix with more time."** Give the limitations.

Questions you will be asked

"Why not just measure the mass with a sensor?" You could, but that is a different problem. The point is that the *controller's own response* already contains the information — no extra hardware needed. It also generalises to disturbances you cannot instrument, like wind.

"Why is your R^2 not higher?" Because K_p and K_d are not uniquely determined — the cost has a flat valley in those directions, so the labels are noisy. K_i , which is uniquely determined and which drives the performance gain, predicts at $R^2 \approx 0.68$.

"How do you know the ML gains are safe?" They are clamped to a validated range before use, and the identification phase always runs on known-stable baseline gains. There is no formal stability proof — that is a stated limitation.

"Why a small network rather than a deep one?" 150 samples and 8 features. A deep network would overfit — and we demonstrated that empirically by comparing four model classes.

"What would you do next?" Extend from altitude to all four channels, replace the fixed 3 s identification window with continuous online adaptation, and add a Lyapunov-based stability guarantee around the learned gains.